

Day10-容器技术与模拟考试

课程目标

- 理解容器基本概念
- 掌握 Podman 容器管理
- 学会构建容器镜像
- 掌握容器存储和网络配置
- 学会配置容器为系统服务
- 完成综合练习

上午 - 容器技术基础

课前故事：容器技术的诞生

为什么需要容器技术？

在容器出现之前，应用部署面临着经典的"依赖地狱"问题：

交付困境：

- 开发环境：应用运行正常
- 测试环境：依赖版本冲突，无法运行
- 生产环境：配置不一致，问题频发

经典台词："在我的机器上能运行！"

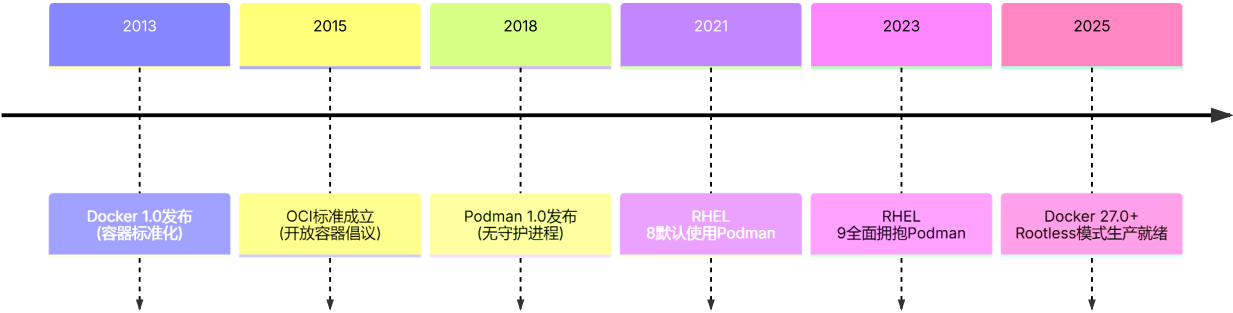
虚拟机时代的问题：

每个应用 = 一个完整虚拟机

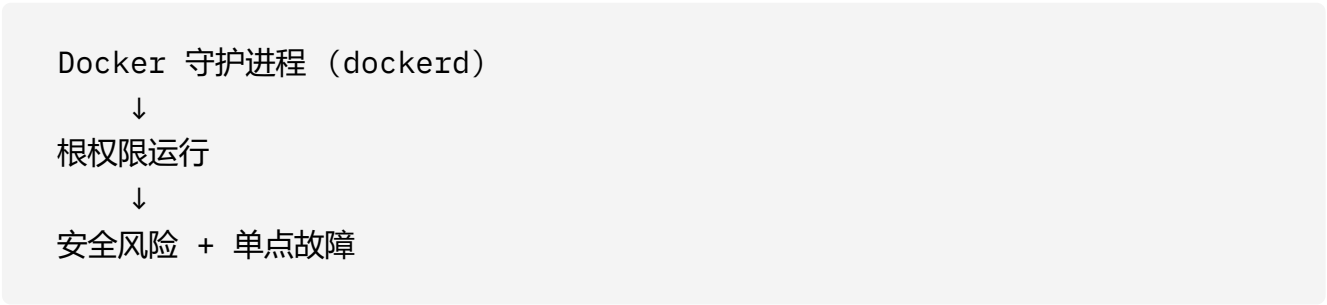
- 启动：等待 5 分钟
- 资源：占用 4GB 内存（应用只需 100MB）
- 维护：每个虚拟机需要独立操作系统更新

容器技术从 Docker 到 Podman 的进化：

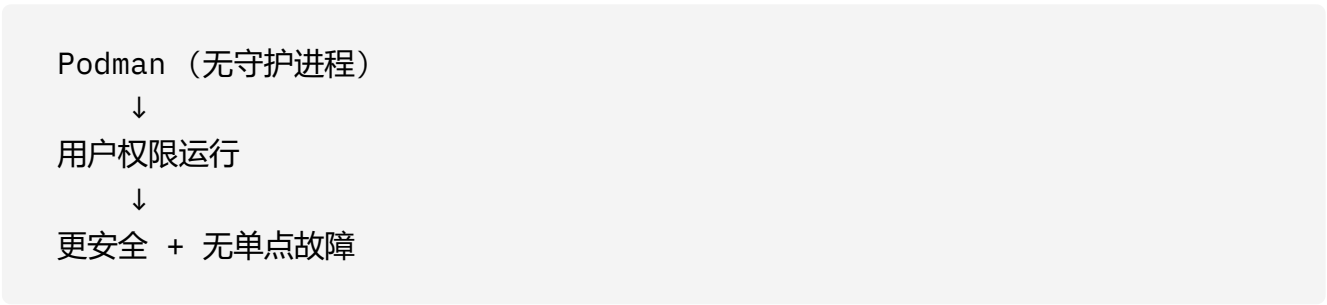
容器技术演进史 (2025更新版)



传统 Docker 模式：



现代 Podman 模式：



Podman vs Docker 对比：

特性	Podman (2025)	Docker (2025)	说明
架构	🔴 Daemonless (无守护进程)	🟡 Daemon-based (守护进程)	Podman更简洁
Rootless支持	✅ 原生支持，设计之初即考虑	⚠️ 27.0+ 才生产就绪	Podman更成熟
用户权限	普通用户即可运行	需要docker组或sudo	Podman更安全
安全性	🔒 高 (无特权守护进程)	🟡 中 (需要dockerd)	Podman优势明显
系统服务集成	✅ 原生systemd支持	⚠️ 需要额外配置	Podman更简单

特性	Podman (2025)	Docker (2025)	说明
兼容性	✅ Docker命令别名兼容	✅ 广泛生态支持	Docker生态成熟
适用场景	🚀 新项目/安全优先	🔄 遗留系统/现有生态	根据需求选择

Podman 的革命性设计：

- 1. 无守护进程架构 (Daemonless)
 - 没有 dockerd 守护进程
 - 直接与容器交互
 - 降低安全风险，无单点故障
- 2. 完全兼容 Docker
 - 命令几乎完全兼容
 - `alias docker=podman` 即可切换
 - 无缝迁移现有工作流
- 3. Rootless 容器优先
 - 普通用户即可运行容器
 - 不需要 sudo 权限
 - UID/GID 自动映射
 - 容器内root → 主机非特权用户
- 4. Pod 管理能力
 - 原生支持 Kubernetes Pod
 - `podman pod` 命令管理
 - 为云原生 (K8s) 做准备

第一部分：容器概念

为什么需要学习容器技术？

通过课前故事，我们了解了容器技术如何解决应用部署的"依赖地狱"问题。
Podman 作为 RHEL 的标准容器工具，提供了更安全、更高效的容器管理方式。

本部分将学习：

- 容器的基本概念和工作原理
- Podman 与 Docker 的区别
- 镜像的管理和使用
- 容器的创建、启动和管理

掌握这些知识，你将能够：

- 使用 Podman 管理容器
- 拉取和运行容器镜像
- 理解容器与虚拟机的区别
- 为云原生应用打下基础

1.1 什么是容器

容器定义：

容器是一个或多个进程的集合，与系统的其他部分隔离

- 轻量级，比虚拟机更高效
- 共享宿主机内核
- 包含应用及其所有依赖项
- 快速启动和停止

容器 vs 虚拟机：

特性	容器	虚拟机
启动速度	秒级	分钟级
资源占用	低	高
隔离性	进程级	系统级
操作系统	共享内核	独立系统

1.2 容器核心技术

RHEL 容器使用的技术：

- **Control Groups (cgroups)** - 资源管理
- **Namespaces** - 进程隔离
- **SELinux** - 安全边界
- **Seccomp** - 安全计算模式

容器工具：

工具	作用
podman	管理容器和镜像
buildah	构建容器镜像
skopeo	检查和复制镜像

1.3 容器相关概念

镜像：

容器的基础，包含应用和所有依赖

- 不可更改（只读）
- 按层构建
- 从 registry 拉取

Registry：

存储和分发镜像的仓库

- **registry.redhat.io** - 红帽官方镜像
- **registry.connect.redhat.com** - 第三方镜像
- **Docker Hub** - 公共镜像

容器：

镜像的运行实例

- 从镜像启动
- 可读写
- 临时性（默认）

相关练习：RH134V9 教案练习

练习名称	对应章节	说明
选择练习：Containers Concepts	第13章	容器概念

第二部分：安装和使用 Podman

2.1 安装容器工具

安装 **container-tools**：

```
# 安装容器工具包组
dnf install container-tools -y

# 查看包含的工具
dnf info container-tools

# 输出示例：
# Summary : A meta-package with container tools such as podman,
#          buildah, skopeo, etc.
```

SHELL

container-tools 包含：

- podman - 容器管理
- buildah - 镜像构建
- skopeo - 镜像操作
- common - 容器监控

2.2 登录 Registry

登录私有仓库：

```
# 登录 registry
podman login registry.lab.example.com

# 输出示例:
# Username: admin
# Password: redhat321
# Login Succeeded!

# 使用非安全连接(证书过期时)
podman login --tls-verify=false registry.lab.example.com

# 指定用户名
podman login -u admin registry.lab.example.com
```

查看登录状态:

```
# 查看当前登录
podman login --get-login registry.lab.example.com

# 输出示例:
# admin
```

2.3 搜索和拉取镜像

搜索镜像:

```
# 搜索镜像
podman search registry.lab.example.com/ubi8

# 输出示例:
# NAME                                DESCRIPTION
# registry.lab.example.com/ubi8/ubi ...
# registry.lab.example.com/ubi8/python-38 ...
```

检查镜像元数据:

```
# 检查镜像信息
# 如果没有 skopeo 可以先安装 dnf install -y skopeo
skopeo inspect docker://registry.lab.example.com/ubi8/ubi:latest

# 输出示例:
# {
#   "Name": "registry.lab.example.com/ubi8/ubi:latest",
#   "Digest": "sha256:...",
#   "RepoTags": ["latest", "8.4"],
#   ...
# }
```

拉取镜像:

```
# 拉取镜像
podman pull registry.lab.example.com/ubi8/ubi:latest

# 输出示例:
# Trying to pull registry.lab.example.com/ubi8/ubi:latest...
# Getting image source signatures
# Copying blob ...
# Writing manifest to image destination
```

查看本地镜像:

```
# 列出本地镜像
podman images

# 或
podman image ls

# 输出示例:
# REPOSITORY                                TAG      IMAGE ID
# registry.lab.example.com/ubi8/ubi         latest
# a33d92f90990 1 hour ago          901 MB
```


容器镜像标签的常见陷阱:

1. 缺少标签导致的问题

```
# X 错误: 没有指定标签
podman pull ubi8/ubi
# 默认使用 latest, 但 latest 不保证存在!

# ✓ 正确: 明确指定标签
podman pull ubi8/ubi:8.4
podman pull registry.lab.example.com/ubi8/ubi:latest
```

SHELL

2. 镜像名称格式

```
# 省略注册表地址
podman pull ubi8/ubi

# 使用完整路径
podman pull registry.lab.example.com/ubi8/ubi:latest
podman pull docker.io/library/nginx:latest
```

SHELL

如果想使用简短的镜像名称, 需要额外配置表

`~/.config/containers/registries.conf`:

```
unqualified-search-registries = ['registry.lab.example.com']
[[registry]]
location = "registry.lab.example.com"
insecure = trueblocked = false
```

SHELL

- 作用: 当你执行 `podman run ubuntu` 时 (可不带完整地址)
- 效果: 自动在这个仓库搜索 `ubuntu` 镜像
- 完整地址变成: `registry.lab.example.com/ubuntu`
- `insecure = true` 的含义:
 - 私有仓库通常自签名证书或用 HTTP
 - 此选项跳过 SSL 验证
 - 仅限内网测试环境使用

第三部分：运行容器

3.1 创建和启动容器

创建容器：

SHELL

```
# 创建容器但不运行
podman create --name mycontainer
registry.lab.example.com/ubi8/ubi sleep infinity

# 输出示例：
#
c54c7ee281581c198cb96b07d78a0f94be083ae94dacbae69c05bd8cd354bbec

# 查看所有容器（包括停止的）
podman ps -a

# 输出示例：
# CONTAINER ID   IMAGE                                COMMAND                  STATUS
NAMES
# c54c7ee28158   .../ubi8/ubi:latest                /bin/bash               Created
mycontainer
```

启动容器：

SHELL

```
# 启动已创建的容器
podman start mycontainer

# 验证
podman ps

# 输出示例：
# CONTAINER ID   IMAGE    COMMAND                  STATUS          NAMES
# c54c7ee28158   .../ubi  /bin/bash               Up 5 seconds    mycontainer
```

3.2 运行容器

podman run 完整选项表：

选项	说明	示例
基本选项		
<code>-d, --detach</code>	后台运行容器	<code>-d</code>
<code>-i, --interactive</code>	保持 STDIN 打开	<code>-i</code>
<code>-t, --tty</code>	分配伪终端	<code>-t</code>
<code>--name NAME</code>	指定容器名称	<code>--name mycontainer</code>
<code>--rm</code>	容器退出时自动删除	<code>--rm</code>
镜像和命令		
<code>IMAGE</code>	镜像名称	<code>registry.lab.example.com/ubi8/ubi</code>
<code>COMMAND</code>	容器启动后执行的命令	<code>sleep infinity</code>
端口映射		
<code>-p, --publish</code>	发布端口到主机	<code>-p 8080:80</code>
<code>-P, --publish-all</code>	发布所有端口	<code>-P</code>
卷挂载		
<code>-v, --volume</code>	挂载卷	<code>-v /host:/container:Z</code>
环境变量		
<code>-e, --env</code>	设置环境变量	<code>-e KEY=value</code>
<code>--env-file</code>	从文件读取环境变量	<code>--env-file env.list</code>
网络		
<code>--network</code>	连接到网络	<code>--network mynet</code>
<code>--add-host</code>	添加主机到 /etc/hosts	<code>--add-host host:ip</code>
资源限制		
<code>-m, --memory</code>	内存限制	<code>-m 512m</code>
<code>--cpu-shares</code>	CPU 份额	<code>--cpu-shares 512</code>
用户和权限		
<code>-u, --user</code>	用户名或 UID	<code>-u 1000</code>
<code>--userns</code>	用户命名空间	<code>--userns keep-id</code>
其他		
<code>--restart</code>	重启策略	<code>--restart always</code>
<code>--privileged</code>	扩展权限	<code>--privileged</code>
<code>--read-only</code>	只读根文件系统	<code>--read-only</code>

直接运行容器：

SHELL

```
# 运行容器 (后台模式)
podman run -d --name mycontainer
registry.lab.example.com/ubi8/ubi sleep infinity

# 参数说明:
# -d = 后台运行 (detach)
# --name = 指定容器名称
# sleep infinity = 保持容器运行

# 运行容器 (交互模式)
podman run -it --name mycontainer
registry.lab.example.com/ubi8/ubi /bin/bash

# 参数说明:
# -i = 交互式 (保持 STDIN 打开)
# -t = 分配伪终端
```

容器状态：

状态	说明
Created	已创建但未启动
Running	正在运行
Stopped	已停止
Paused	已暂停
Exited	已退出

3.3 容器交互

在容器中执行命令：

在运行中的容器执行命令

```
podman exec mycontainer ps -ax
```

输出示例:

```
# PID TTY          TIME CMD
```

```
# 1 ?           0:00 /usr/bin/coreutils --coreutils-prog-  
shebang=sleep /usr/bin/sleep infinity
```

```
# 15 ?          0:00 ps -ax
```

交互式进入容器

```
podman exec -it mycontainer /bin/bash
```

参数说明:

-it = 交互式 + 伪终端

查看容器日志:

查看容器日志

```
podman logs mycontainer
```

实时查看日志

```
podman logs -f mycontainer
```

相关练习

练习名称	对应章节	说明
练习: Deploy Containers	第13章	部署容器

第四部分：构建容器镜像

从RHCSA考纲移除

4.1 ContainerFile

什么是 **ContainerFile**:

文本文件，包含构建容器镜像的指令

```
FROM registry.lab.example.com/ubi8/ubi:latest
RUN dnf install -y python3
CMD ["/bin/bash", "-c", "sleep infinity"]
```

指令说明：

指令	说明	示例
FROM	基础镜像	FROM registry.access.redhat.com/ubi8/ubi:latest
RUN	执行命令	RUN dnf install -y python3
CMD	容器启动时执行的默认命令	CMD ["/bin/bash", "-c", "sleep infinity"]
ENTRYPOINT	容器启动入口点	ENTRYPOINT ["/bin/myapp"]
COPY	复制文件到容器	COPY myfile.txt /app/
ADD	复制文件/URL/tar 到容器	ADD archive.tar.gz /app/
ENV	设置环境变量	ENV APP_ENV=production
ARG	构建时变量	ARG VERSION=1.0
WORKDIR	工作目录	WORKDIR /app
USER	切换用户	USER appuser
VOLUME	声明挂载点	VOLUME /data
EXPOSE	声明端口	EXPOSE 8080
LABEL	元数据标签	LABEL version="1.0"

CMD vs ENTRYPOINT：

特性	CMD	ENTRYPOINT
可被覆盖	✔ 是	✗ 否（除非 --entrypoint）
可追加参数	✔ 是	⚠ 需要 CMD 配合
推荐用途	默认命令/参数	主入口程序

组合使用示例：

```
ENTRYPOINT ["/bin/myapp"]
CMD ["--default-arg"]
# 运行时: /bin/myapp --default-arg
# 覆盖: podman run myimage --custom-arg
```

4.2 构建镜像

从考纲移除

使用 **ContainerFile** 构建:

```
# 创建构建目录
mkdir myimage
cd myimage

# 创建 ContainerFile
cat > Containerfile << EOF
FROM registry.lab.example.com/ubi8/ubi:latest
RUN dnf install -y python3
CMD ["/bin/bash", "-c", "sleep infinity"]
EOF

# 构建镜像
podman build -t myimage:1.0 .

# 参数说明:
# -t myimage:1.0 = 指定镜像名称和标签
# . = 构建上下文 (当前目录)

# 输出示例:
# STEP 1/3: FROM registry.lab.example.com/ubi8/ubi:latest
# STEP 2/3: RUN dnf install -y python3
# STEP 3/3: CMD ["/bin/bash", "-c", "sleep infinity"]
# COMMIT myimage:1.0 -> 35ab820880f
# Successfully tagged localhost/myimage:1.0
```

SHELL

验证镜像:

```
# 查看镜像
podman images | grep myimage

# 检查镜像详情
podman inspect localhost/myimage:1.0
```

操作演示：容器基础

演示目标

掌握 Podman 基本操作

演示步骤

任务1：查看镜像

```
# 列出本地镜像
podman images

# 搜索镜像
podman search registry.lab.example.com/ubi8
```

任务2：运行容器

```
# 运行容器
podman run -d --name test ubi:8 sleep infinity

# 查看运行中的容器
podman ps

# 查看所有容器
podman ps -a
```

任务3：容器交互

SHELL

```
# 在容器中执行命令
podman exec test hostname

# 查看容器日志
podman logs test
```

相关练习：

练习名称	对应章节	说明
练习：Deploy Containers	第13章 P415	部署容器(含构建镜像)

下午 - 容器高级管理

第五部分：容器存储

5.1 持久存储

为什么需要持久存储：

- 容器默认存储是临时的
- 删除容器后数据丢失
- 需要持久化数据

挂载卷：

```
# 挂载主机目录到容器
podman run -d --name mycontainer \
  -v /host/path:/container/path:Z \
  registry.lab.example.com/ubi8/ubi sleep infinity

# 参数说明:
# -v = 卷挂载
# /host/path = 主机目录
# /container/path = 容器内路径
# :Z = 自动设置 SELinux 上下文 (私有)
# :z = 共享 SELinux 上下文
```

SELinux 上下文选项:

选项	说明	使用场景
:Z	私有标签, 容器独占访问	单个容器使用
:z	共享标签, 多容器共享	多个容器共享同一卷

挂载卷行为说明:

源类型	源为空	源非空
数据卷	容器目录内容复制到卷	卷内容覆盖容器目录
主机目录	若容器目录被 VOLUME 声明, 则复制	主机目录内容覆盖容器目录

Rootless 容器 UID 映射:

```
容器 UID/GID → 主机 UID/GID
0 (root)      → 用户 UID (如 1000)
1             → 100000
27            → 100026
...
```

查看 UID 映射:

```
# 查看容器内的 UID 映射
podman unshare cat /proc/self/uid_map

# 输出示例:
# 0 1000 65536
# 100000 1 65536
```

5.2 环境变量

传递环境变量：

```
# 使用 -e 传递环境变量
podman run -d --name mydb \
  -e MYSQL_USER=student \
  -e MYSQL_PASSWORD=redhat \
  -e MYSQL_DATABASE=mydata \
  registry.lab.example.com/rhel8/mariadb-105

# 查看环境变量
podman exec mydb env | grep MYSQL
```

第六部分：容器网络

6.1 端口映射

映射端口：

```
# 映射容器端口到主机
podman run -d --name myserver \
  -p 8080:80 \
  registry.lab.example.com/ubi8/httpd-24

# 参数说明:
# -p 8080:80 = 主机端口8080映射到容器端口80

# 查看端口映射
podman port myserver

# 输出示例:
# 80/tcp -> 0.0.0.0:8080
```

开放防火墙端口：

```
# 开放端口
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --reload
```

⚠ 易错点：端口映射和防火墙

容器端口映射的常见陷阱：

1. 端口映射格式

```
# ✗ 错误：格式错误
podman run -p 80:8080 myimage # 容器:主机（反了！）

# ✓ 正确：主机端口:容器端口
podman run -p 8080:80 myimage # 8080是主机，80是容器
```

2. 忘记开放防火墙

```
# X 错误：映射端口但忘记防火墙
podman run -p 8080:80 httpd
# 外部无法访问！

# ✔ 正确：映射 + 防火墙
podman run -p 8080:80 httpd
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --reload
```

3. 主机端口冲突

```
# X 错误：端口已被占用
podman run -p 80:80 httpd
# Error: container create failed: error forwarding port

# ✔ 正确：检查端口占用或使用其他端口
ss -tlnp | grep :80
podman run -p 8080:80 httpd
```

记忆技巧：

- **-p 外:内** - 外面是主机，里面是容器
- 映射后注意防火墙配置！

6.2 容器网络

创建网络：

```
# 创建带 DNS 的网络
podman network create --subnet 10.89.0.0/24 mynet

# 查看网络
podman network ls

# 输出示例:
# NETWORK ID      NAME      DRIVER
# 2f259bab93aa    podman    bridge
# fe680efc5276    mynet     bridge

# 检查网络详情
podman network inspect mynet
```

连接容器到网络：

```
# 运行容器并连接到网络
podman run -d --name container1 \
  --network mynet \
  registry.lab.example.com/ubi8/ubi sleep infinity

# 连接已有容器到网络
podman network connect mynet container2
```

相关练习

练习名称	对应章节	说明
练习：Manage Container Storage and Network Resources	第13章 P431	管理容器存储和网络资源

第七部分：容器作为系统服务

7.1 systemd 服务

为什么使用 **systemd**：

- 容器随系统自动启动

- 统一的服务管理（systemctl start/stop/restart）
- 自动重启失败容器
- 与系统服务一致性管理
- 支持依赖关系配置

RHEL 9 Systemd集成最佳实践：

根据 [RHEL 9官方文档](#)，Podman与systemd的集成提供了完整的容器生命周期管理：

systemd服务类型：

服务类型	路径	权限	适用场景
系统服务	/etc/systemd/system/	root	全局容器，需要特权访问
用户服务	~/.config/systemd/user/	普通用户	用户容器，rootless模式

podman generate systemd 选项表：

选项	说明	行为
--name, -n	容器名称	指定要生成服务的容器
--files, -f	生成文件到当前目录	生成 .service 文件
--new	每次启动创建新容器	停止时删除容器
--restart-policy	重启策略	no, on-failure, always

--new vs 无 --new 对比：

特性	无 --new	使用 --new
启动时	podman start (现有容器)	podman run (创建新容器)
停止时	podman stop	podman stop + podman rm
容器删除	✗ 否	✓ 是
适用场景	持久化容器	临时容器

生成 systemd 服务文件：

```
# 为现有容器生成 systemd 服务
podman generate systemd --name mycontainer --files

# 参数说明:
# --name = 容器名称
# --files = 生成到当前目录
# --new = 每次启动创建新容器

# 输出示例:
# /home/user/container-mycontainer.service
```

生成的服务文件内容:

```
# container-mycontainer.service
[Unit]
Description=Podman container-mycontainer.service
After=network-online.target
Wants=network-online.target

[Service]
Type=notify
Restart=on-failure
ExecStartPre=/usr/bin/rm -f %t/%n.ctr-id
ExecStart=/usr/bin/podman run --cidfile=%t/%n.ctr-id --
cgroups=no-common --rm -d --name mycontainer ...
ExecStop=/usr/bin/podman stop --ignore --cidfile=%t/%n.ctr-id
ExecStopPost=/usr/bin/podman rm -f --ignore --cidfile=%t/%n.ctr-
id

[Install]
WantedBy=multi-user.target
```

7.2 配置用户服务

安装服务文件:


```
# 创建用户服务目录
mkdir -p ~/.config/systemd/user

# 移动服务文件
mv container-mycontainer.service ~/.config/systemd/user/

# 重新加载 systemd
systemctl --user daemon-reload
```

启用和启动服务：

```
# 启用服务（开机自启）
systemctl --user enable container-mycontainer.service

# 启动服务
systemctl --user start container-mycontainer.service

# 查看状态
systemctl --user status container-mycontainer.service

# 输出示例：
# • container-mycontainer.service - Podman container-
  mycontainer.service
#    Loaded: loaded (/home/user/.config/systemd/user/...)
#    Active: active (running) since ...
```

7.3 系统启动时启动

启用 **linger**：

```
# 让用户服务在系统启动时运行
loginctl enable-linger

# 验证
loginctl show-user $USER

# 输出示例：
# Linger: yes
```

禁用 linger:

```
# 禁用 linger
loginctl disable-linger
```

SHELL

操作演示：容器高级管理

演示目标

掌握容器存储、网络和服务配置

演示步骤

任务1：持久存储

```
# 创建数据目录
mkdir ~/mydata

# 运行容器并挂载目录
podman run -d --name test \
  -v ~/mydata:/data:Z \
  ubi:8 sleep infinity

# 验证挂载
podman exec test ls /data
```

SHELL

任务2：端口映射

```
# 运行容器并映射端口
podman run -d --name web \
  -p 8080:80 \
  ubi:8/httpd-24

# 查看端口
podman port web
```

SHELL

任务3：配置用户级系统服务

SHELL

```
# 生成服务文件
podman generate systemd --name test --files

# 安装服务
mv container-test.service ~/.config/systemd/user/

# 启用服务
systemctl --user daemon-reload
systemctl --user enable --now container-test.service
loginctl enable-linger
```

相关练习：

练习名称	对应章节	说明
练习：Managing Containers as System Services	第13章 P442	将容器作为系统服务管理
总练习：Running Containers	第13章 P448	运行容器综合练习

今日总结

命令清单

Podman：

```
podman search      # 搜索镜像
podman pull        # 拉取镜像
podman images      # 列出镜像
podman run         # 运行容器
podman ps          # 列出容器
podman exec        # 执行命令
podman logs        # 查看日志
podman build       # 构建镜像
podman generate systemd # 生成服务文件
```

Systemd 用户服务：

```
systemctl --user daemon-reload # 重新加载
systemctl --user enable        # 启用服务
systemctl --user start         # 启动服务
loginctl enable-linger         # 开机启动
```

常见任务

1. 构建容器镜像

```
# 创建 ContainerFile
cat > Containerfile << EOF
FROM registry.lab.example.com/ubi9/ubi:latest
RUN dnf install -y python3
CMD ["/bin/bash", "-c", "sleep infinity"]
EOF

# 构建镜像
podman build -t myapp:1.0 .
```

2. 配置容器服务

```
# 运行容器
podman run -d --name myapp -v /data:/app:Z myapp:1.0

# 生成服务
podman generate systemd --name myapp --new --files

# 安装服务
mv container-myapp.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user enable --now container-myapp.service
loginctl enable-linger
```

本日总结

关键命令速查

命令	作用	常用选项
podman search	搜索镜像	registry
podman pull	拉取镜像	镜像名
podman images	列出镜像	无
podman run	运行容器	-d, -it, --name
podman ps	列出容器	-a
podman exec	执行命令	-it
podman build	构建镜像	-t
podman generate systemd	生成服务	--name, --new, --files
systemctl --user	用户服务管理	enable, start
loginctl enable-linger	开机启动	用户名

参考资料与延伸阅读

官方文档

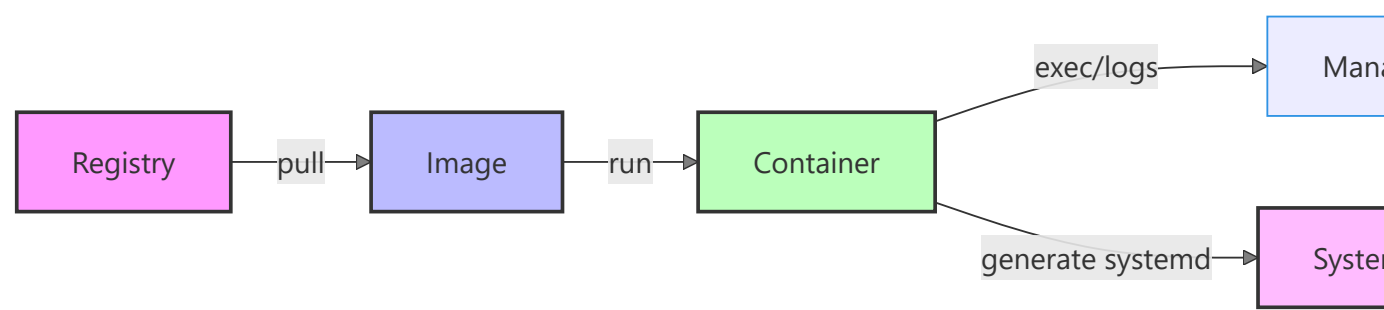
资源	链接	说明
RHEL 9 容器文档	Building, running, and managing containers	RHEL 9 官方容器管理指南
Podman 文档	Podman.io	Podman 官方文档
Buildah 文档	Buildah.io	Buildah 镜像构建工具
Skopeo 文档	GitHub - Skopeo	Skopeo 镜像操作工具

社区资源

资源	链接	说明
Podman vs Docker 2025	Docker vs Podman: An In-Depth Comparison (2026)	2025-2026年深度对比
Podman Rootless 实践	Rootless containers with Podman: The basics	Rootless 容器基础
Systemd 集成	Configure a container to start automatically as a systemd service	容器自动启动配置

技术要点回顾

容器核心概念：



技能掌握清单：

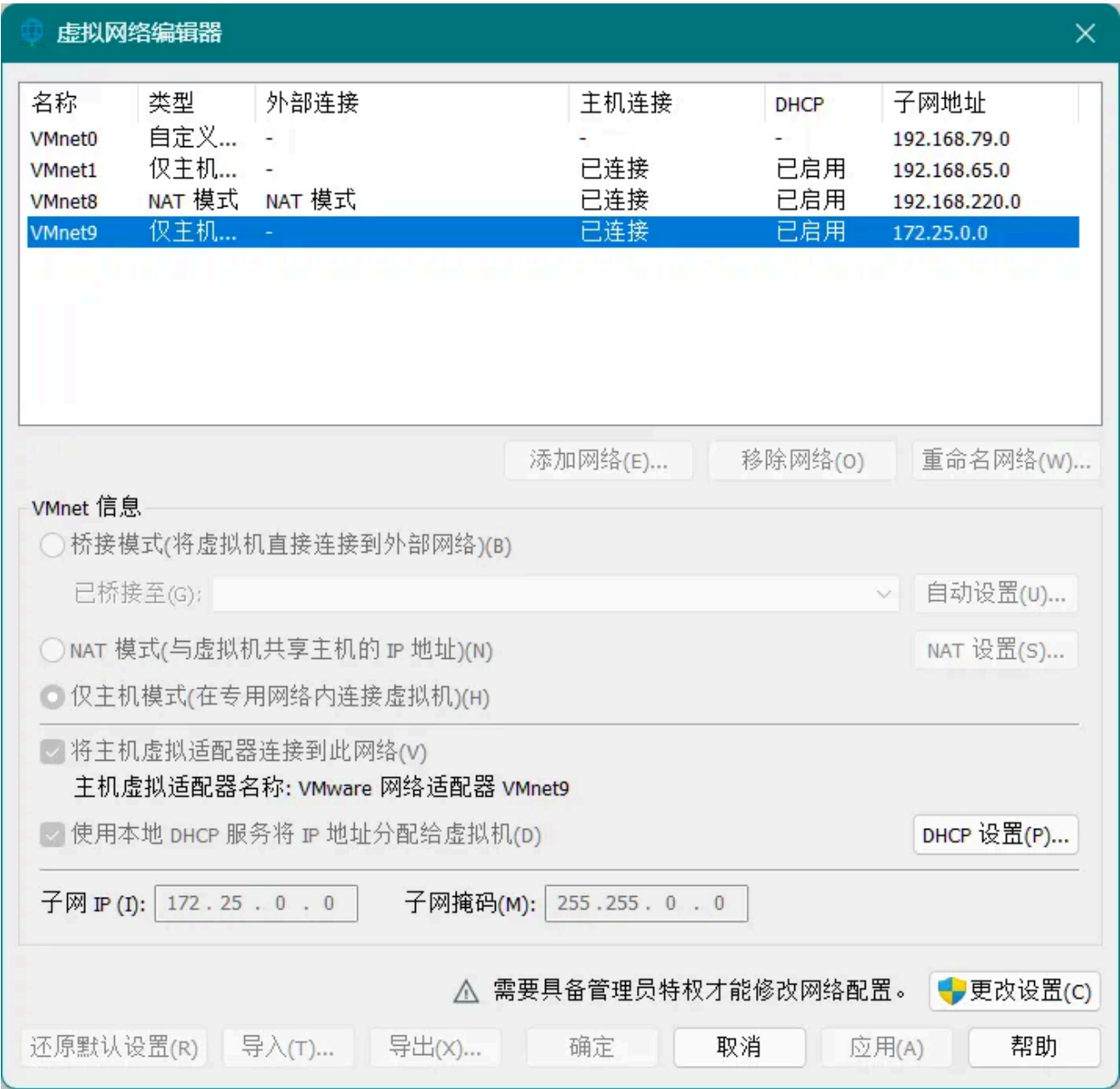
- ☒ 使用 Podman 搜索和拉取镜像 ☒ 2026-02-27
- ☒ 运行和管理容器 ☒ 2026-02-27
- ☐ 编写 Containerfile 构建镜像(不考)
- ☒ 配置容器持久化存储 ☒ 2026-02-27
- ☒ 配置容器网络和端口映射 ☒ 2026-02-27

- ✓ 将容器配置为 systemd 服务 ✓ 2026-02-27
- ✓ 配置容器开机自动启动 ✓ 2026-02-27
- ✓ 排查容器常见问题 ✓ 2026-02-27

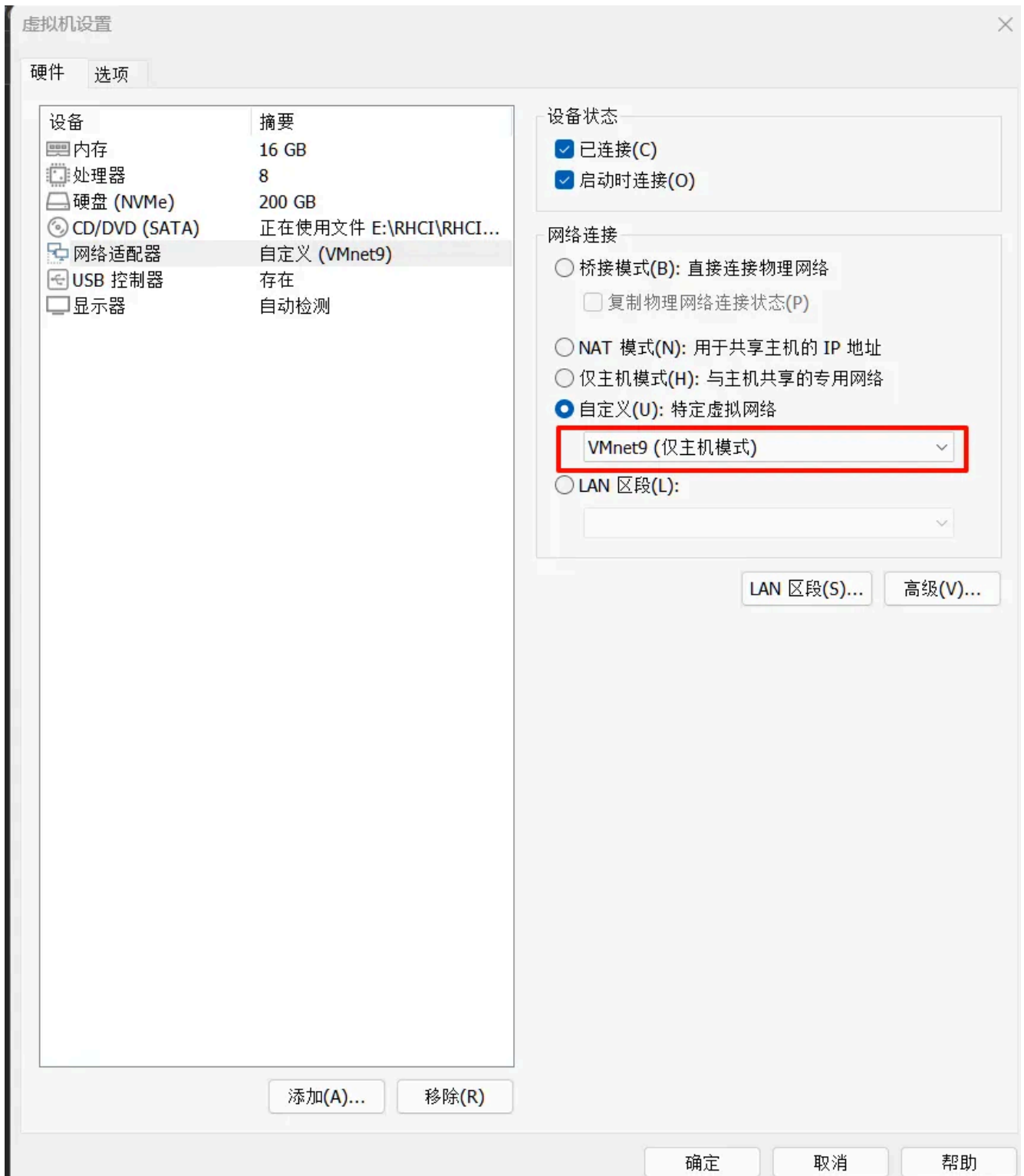
考前辅导

模拟考试环境准备

虚拟机创建仅主机网络(子网为172.25.0.0/16, 最好禁用dhcp):



虚拟机关机情况下, 进入虚拟机设置中的网络适配器, 添加自定义虚拟网络为上述添加的仅主机虚拟网络:



1. 如何上传文件到虚拟机

方法一：通过终端使用 **scp** 上传文件

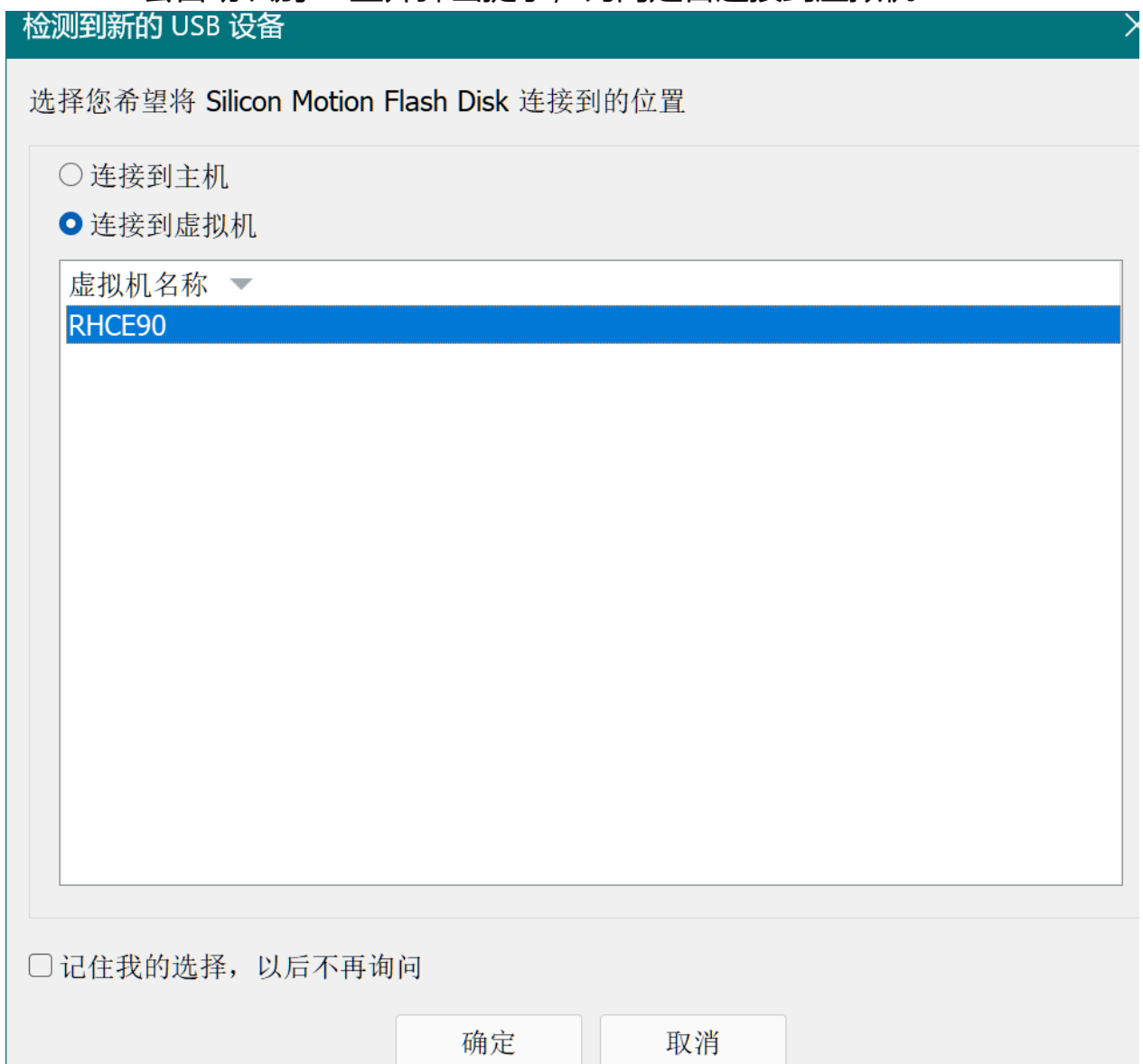

```
# 确保能够使用 SSH 连接虚拟机
ssh kiosk@172.25.250.10 hostname
# 可能需要输入密码后登录

# 测试通过后
# 上传单个文件的方法
scp /local/path/file.tar.gz kiosk@172.25.250.10:/home/kiosk/

# 上传整个目录的方法
scp -r /local/path/directory kiosk@172.25.250.10:/home/kiosk/
```

方法二：通过 U 盘上传（VMware 虚拟机）

1. 首先确保虚拟机处于运行状态
2. 在物理机上插入 U 盘，将所需文件复制到 U 盘
3. VMware 会自动识别 U 盘并弹出提示，询问是否连接到虚拟机



4. 选择连接后，登录虚拟机，U 盘会自动挂载到 `/run/media/<user>/` 目录

5. 复制文件到目标位置：

SHELL

```
# 查看 U 盘挂载点
ls /run/media/kiosk/

# 复制文件
cp /run/media/kiosk/USB_DISK/file.tar.gz /home/kiosk/
cp /run/media/kiosk/USB_DISK/Desktop.tar.gz /home/kiosk/
```

方法三：使用 SFTP 客户端工具

工具名称	特点	下载地址
FinalShell	免费、一体化的SSH工具，自带SFTP	https://www.hostbuf.com/
MobaXterm	免费Home Edition，自带SFTP	https://mobaxterm.mobatek.net/
XShell/XFtp	商业软件，专业SSH/SFTP工具	https://www.netsarang.com/

基本操作步骤（以 FinalShell 为例）：

1. 安装并打开 FinalShell
2. 新建SSH连接，输入虚拟机IP（172.25.250.10）
3. 输入用户名（kiosk）和密码
4. 连接成功后，底部会自动显示SFTP文件管理器
5. 直接拖拽文件到 `/home/kiosk/` 目录即可上传

其他工具类似操作：

- MobaXterm：连接后在左侧会看到SFTP窗口
- XShell + XFtp：XShell中可直接打开XFtp进行文件传输

2. 重置 RH134 教学环境

在 foundation0 上执行：

重置练习环境

```
[kiosk@foundation0 ~]$ rht-clearcourse 0
```

切换到 CSA 练习环境

```
[kiosk@foundation0 ~]$ rht-setcourse rh134
```

重置所有虚拟机（等待加载启动完成）

```
[kiosk@foundation0 ~]$ rht-vmctl reset -y all
```

启动 classroom（必做！start all 不会包含 classroom）

```
[kiosk@foundation0 ~]$ rht-vmctl reset -y classroom
```

验证虚拟机启动状态：

- 打开 Virtual Machine Manager 界面查看
- classroom、servera、serverb 负载下降即表示启动完成

3. 安装模拟考试环境

将 **resources/csa模拟环境资源包.rar** 压缩包解压(解压密码同课程密码)，解压后得到三个 **tar.gz** 文件，将这三个文件分别上传到虚拟机的kiosk用户家目录下。

上传并解压 **ansible.tar.gz**：

上传到 /home/kiosk/ 目录

```
[kiosk@foundation0 ~]$ tar zxvf ansible.tar.gz
```

```
[kiosk@foundation0 ~]$ cd ansible/
```

```
[kiosk@foundation0 ansible]$ bash setup.sh
```

等待执行结束

4. 上传评分脚本

上传并解压 **bin.tar.gz**：

上传到 /home/kiosk/

```
[kiosk@foundation0 ~]$ tar zxvf bin.tar.gz
```

```
[kiosk@foundation0 ~]$ chmod u+x bin/exam-grade
```

5. 上传考题

上传并解压 **Desktop.tar.gz**:

```
# 上传到 /home/kiosk/  
[kiosk@foundation0 ~]$ tar zxvf Desktop.tar.gz
```

SHELL

6. 创建系统快照

完成环境配置后:

1. 关闭虚拟机
2. 创建系统快照 (保存当前干净状态)

刷题流程

刷题前准备

1. 还原快照, 登录 foundation0
2. 启动所有虚拟机 (包括 classroom)
3. 等待 **10** 分钟, 确保所有虚拟机就绪

启用考试计时 (可选)

```
[kiosk@foundation0 ~]$ su  
Password: Asimov  
[root@foundation0 ~]# touch /content/RHCSA.html  
[root@foundation0 ~]# exit  
[kiosk@foundation0 ~]$
```

SHELL

答题顺序建议

优先完成首道题目：

题目	机器	内容
网络配置	node1/servera	配置网络连接
root 密码	node2/serverb	修改 root 密码

然后完成其他题目

验证与评分

完成后执行判分：

```
[kiosk@foundation0 ~]$ bin/exam-grade
```

SHELL

重要提醒：

- 真正考试时没有判分脚本！
- 做完每道题要掌握如何验证结果
 - 离开考场前一定要重启检查
 - 熟悉常用文档的查找方法
 - 平时多试错，考试遇问题不慌张

重新刷题

1. 恢复快照
 2. 重新开始
-

评分标准

项目	分数
满分	300 分
及格分	210 分

答题建议

不要死记硬背答案！
不要仅依赖判分脚本！

真正考试时：

- 没有判分脚本
- 需要自己验证结果
- 需要自己查找文档

恭喜完成课程！

祝学业顺利！ 🐧

最后祝大家认证考试取得好成绩！

— End of Day 10 —

Delucia - 2025-01-15